COMPLETE GUIDE

7 Agentic AI Design Patterns

Industrial agentic AI design patterns that help you build real, authentic, and powerful AI systems

AI Agents | Design Patterns | LLM Workflows | Multi-Agent | Tool Use

Naved Khan
AI Engineer

Follow

All Patterns

7 Agentic AI Design Patterns

1. **Agentic RAG**
   Retrieval + reasoning for context-aware answers

2. **Self-Reflection**
   Agents critique and improve their own outputs

3. **Multi-Agent Workflow**
   Specialized agents collaborating to solve tasks

4. **Modern Tool Use**
   Agents leveraging APIs, databases, and apps

5. **ReAct Agent**
   Combining reasoning with real-world actions

6. **CodeAct Agent**
   Writing & executing code for dynamic problem solving

7. **Plan-and-Execute**
   Structured planning before execution

Naved Khan
AI Engineer

Follow

1 Agentic RAG Pattern

Agentic RAG makes AI smarter at finding answers. It searches multiple sources and connects ideas. The agent adapts its search strategy based on what it finds. This helps it answer complex questions accurately.

Key Components

- **Retrieval Agent** - Manages how information is searched
- **Knowledge Base** - Stores documents and data
- **Reasoning Engine** - Connects ideas from different sources
- **Context Integrator** - Combines all found information
- **Query Planner** - Breaks complex questions into smaller searches
- **Reflection Module** - Checks if answers are good enough

Flow

Query Planner → Retrieval Agent → Knowledge Base → Reasoning Engine → Context Integrator → Reflection Module

Naved Khan
AI Engineer

Follow

2 Self-Reflection Pattern

Self-reflection helps AI improve its own work. The agent creates something first. Then it reviews what it made and finds problems. It fixes those problems and tries again. This cycle repeats until the work is good enough.

Key Components

- **Primary Agent** - Creates the initial output
- **Critic Agent** - Reviews and finds mistakes
- **Feedback Loop** - Sends improvements back to the creator
- **Iteration Controller** - Manages how many times to try
- **Quality Validator** - Checks if work meets standards
- **Refinement Engine** - Makes improvements based on feedback

Flow

Primary Agent → Critic Agent → Quality Validator → Refinement Engine → Final Output

(loop back to Critic Agent if refinement needed)

Naved Khan
AI Engineer

Follow

3 Multi-Agent Workflow Pattern

> Multi-agent systems use many specialized AI helpers. Each helper is good at one thing. They work together to solve big problems. Some work at the same time. Others wait for results from teammates. A coordinator manages everyone.

Key Components

- **Coordinator Agent** - Organizes the whole workflow
- **Specialist Agents** - Experts in different areas
- **Communication Protocol** - How agents talk to each other
- **Task Allocator** - Assigns work to the right agents
- **Result Aggregator** - Combines outputs from all agents
- **State Manager** - Tracks progress and agent status
- **Conflict Resolver** - Fixes disagreements between agents

Flow

Coordinator Agent → Task Allocator → Specialist Agents → Inter-Agent Communication → Result Aggregator → Final Integration

Naved Khan
AI Engineer

Follow

4 Modern Tool Use Pattern

Modern tool use lets AI interact with real systems. The agent sees what tools are available. It picks the right tools for the job. Then it uses them to get information or take actions. This makes AI much more powerful.

Key Components

- **LLM** - Decides which tools to use
- **Tool Registry** - List of all available tools
- **Tool Executor** - Runs the tools and handles errors
- **Result Integrator** - Uses tool results in reasoning
- **Function Schema Manager** - Keeps tool definitions updated
- **Tool Selector** - Chooses relevant tools from the list
- **Error Handler** - Manages when tools fail

Flow

LLM → Tool Selector → Tool Registry → Tool Executor → Result Integrator → Response Generation

Naved Khan
AI Engineer

Follow

5 ReAct Agent Pattern

> ReAct mixes thinking and doing together. The agent thinks about what to do next. Then it does it and sees what happens. It uses those results to think again. This cycle continues until the task is done.

Key Components

- **LLM** - The main reasoning engine
- **Tool Interface** - Connects to external tools
- **Observation Handler** - Processes tool results
- **State Manager** - Keeps track of conversation and progress
- **Reasoning Engine** - Generates thoughts and plans
- **Action Executor** - Performs selected actions
- **Loop Controller** - Manages the think-act cycle

Flow

LLM → Reasoning Engine → Action Executor → Observation Handler → Reflection → Final Response

(loop back to Reasoning Engine if task not complete)

Naved Khan
AI Engineer

Follow

6 CodeAct Agent Pattern

> CodeAct agents write code to solve problems. They analyze what needs to be done. Then they write Python or Bash code to do it. They run the code and see what happens. If there are errors, they fix them and try again.

Key Components

- **LLM** - Generates code based on the problem
- **Code Generator** - Creates Python or Bash code
- **Code Executor** - Runs code in a safe environment
- **Execution Environment** - Maintains state across code runs
- **Error Handler** - Processes code errors and exceptions
- **State Manager** - Tracks variables and execution state
- **Result Observer** - Captures code execution outputs

Flow

LLM → Code Generator → Code Executor → Result Observer → Error Handler → Final Solution

Naved Khan
AI Engineer

Follow

7 Plan-and-Execute Pattern

Plan and execute separates planning from doing. First, the agent creates a complete plan with all steps. Then it executes each step one by one. This works well for tasks with clear steps and dependencies. It is more efficient than mixing planning and doing.

Key Components

- **Planner LLM** - Creates the execution plan
- **Executor LLM** - Performs individual plan steps
- **Plan Store** - Saves and manages the plan
- **Execution Engine** - Runs steps one by one
- **Plan Validator** - Checks if plan is complete
- **Revision Controller** - Updates plan based on results
- **Result Integrator** - Combines step outputs

Flow

Planner LLM → Plan Validator → Plan Store → Execution Engine → Result Integrator → Final Validation → Task Completion

Naved Khan
AI Engineer

Follow

Thank You!

Naved Khan

AI Engineer

Follow for more AI insights and tech content

Like Comment Share Save